

CampusFlow Web Service Report

Name: ZHU FUXIN / シュフシン

Student ID: M25W7195

Assignment: Web service client, account authorization, Docker, and Azure deployment

1. Project Overview

CampusFlow is a desktop browser client backed by a Spring Boot web service. The project combines a designed frontend with external information services, database persistence, account authorization, and container deployment.

The final client is organized as three clearly separated pages:

1. Today is the default page. It displays Kyoto preset content immediately, then updates weather and country information from the backend.
2. Portfolio is a public personal page for photography, drawing, design, and project work. Guest data is available before login.
3. Account supports local registration and login. Google and GitHub are optional third-party login methods.

English, Japanese, and Simplified Chinese can be switched from the same language control. The visual system is shared by all three pages, while the composition of each page is intentionally different.

2. System Architecture

The application uses:

- HTML, CSS, JavaScript, and GSAP for the browser client
- Java 17 and Spring Boot for the web service
- Spring Security for OAuth, CSRF, and password encoding
- MySQL 8 for the local Docker database
- H2 for automated regression tests and the stateless Azure runtime
- Docker and Docker Compose for reproducible local execution
- Docker Hub and Azure App Service for the online container

The browser and API are served from one Spring Boot application. This avoids a separate CORS configuration and keeps the classroom deployment easy to demonstrate.

3. Main Web Services

Daily information

GET /api/home receives a country and city, calls REST Countries and Open-Meteo, and returns one combined response. The weather request uses the timezone returned by geocoding, so the displayed date and time belong to the selected city rather than the server timezone.

Profile and portfolio

GET /api/profile returns a public profile. Student ID and phone are excluded for visitors. An authenticated owner can update their profile through

POST /api/profile.

GET /api/portfolio returns either the four preset guest works or the current user's portfolio. Authenticated users can create, update, and delete only their own items.

Accounts

POST /api/register creates a normal **USER** account. **POST /api/authenticate** verifies a local password and issues an HttpOnly session cookie. **GET /api/auth/me** reports the current public account state, and **POST /api/auth/logout** removes the session.

Google and GitHub OAuth use the same account model. After authorization, the provider name, display name, and avatar can be shown in the Account and Portfolio pages.

An **ADMIN** account is never created by public registration. It is provisioned only through server environment variables and uses **/api/admin/users** to review or disable accounts.

4. Security Design

- Local passwords are encoded with BCrypt strength 12.
- Passwords are never returned by an API.
- Password fields are masked in the browser.
- The session token is stored in an HttpOnly cookie, not browser local storage.
- Azure enables the Secure cookie flag.
- State-changing API calls include a CSRF token.
- Server-side ownership checks protect profile and portfolio writes.

- Public registration cannot request the administrator role.
- OAuth secrets and administrator credentials are environment variables.

5. API Summary

Method	Path	Purpose
GET	<code>/api/home</code>	Combined weather and country data
GET	<code>/api/profile</code>	Public or current-owner profile
POST	<code>/api/profile</code>	Save the signed-in owner's profile
GET	<code>/api/portfolio</code>	Public or current-owner works
POST	<code>/api/portfolio</code>	Create an owned work
PUT	<code>/api/portfolio/{id}</code>	Update an owned work
DELETE	<code>/api/portfolio/{id}</code>	Delete an owned work
POST	<code>/api/register</code>	Create a normal local account
POST	<code>/api/authenticate</code>	Sign in with username and password
GET	<code>/api/auth/me</code>	Read current account state
POST	<code>/api/auth/logout</code>	Sign out
GET	<code>/api/oauth/status</code>	Read third-party authorization state
GET	<code>/api/admin/users</code>	Administrator account management

Interactive API documentation is available at:

```
http://localhost:8080/api-docs.html
```

6. Docker Deployment

The Dockerfile uses a multi-stage build. Maven compiles the project and runs the automated test suite before the runtime image is produced.

Local Docker Compose starts:

- `campus-flow-app` on port `8080`
- `campus-flow-mysql` as the MySQL sidecar

The application connects to MySQL with the Compose service name `mysql`.

Secrets can be supplied from a local `.env` file, which is excluded from Git.

Run the complete local system with:

```
docker compose up -d --build
```

Then open:

```
http://localhost:8080/index.html
```

7. Azure Deployment

The final image is published as:

```
berhish/campus-flow:release-20260724
```

The only Azure target is:

```
Web App: campusflow-final-0724
URL: https://campusflow-final-0724-grbzdrzczdyhyea.japanwest-01.azurewebsites.net
Container port: 8080
```

Azure uses the H2 runtime profile because the Free F1 plan does not provide a MySQL sidecar. The same API contracts, account model, and frontend are used. Google and GitHub callback URLs are derived from **APP_BASE_URL**.

8. Verification

The automated regression suite checks:

- public registration cannot create an administrator
- local passwords are BCrypt hashes
- an old signing secret cannot authorize an administrator request
- unauthenticated profile writes are rejected
- authenticated profile saves succeed
- guest portfolio contains four distinct images
- private profile fields are hidden from visitors
- English, Japanese, and Chinese are included in the release UI
- removed classroom/debug labels cannot reappear
- Google and GitHub redirects use the configured public base URL
- weather uses the selected city's timezone

The final browser acceptance also checks page layout, guest presets, masked password input, language switching, profile/portfolio ownership, and account state.

9. Conclusion

CampusFlow demonstrates a complete browser client and Spring Boot web service with external data, secure accounts, Docker, and one Azure container.